

COURSE CODE/TITLE: CPE 010-CPE21S1 - Data Structures and Algorithms

SECTION: CPE21S1

DATE PERFORMED: August 4 2026

DATE SUBMITTED: August 4 2026

NAME: Mendoza, Roy Gabriel I.

INSTRUCTOR: Engr. Jimlord Quejado

ACTIVITY NO. 5.1 Progress Report Hands-on Activity 5.1 Queues

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A:

CODE:

```
//  
// Created by royga on 8/4/2026.  
//  
#include <iostream>  
#include <queue>  
#include <string>  
  
//prototyping  
void display(std::queue<char> r);  
  
int main() {  
    //pushing  
    std::queue<char> q;  
    q.push('c');  
    q.push('p');  
    q.push('e');  
    q.push('0');  
    q.push('1');  
    q.push('0');  
  
    //display  
    std::cout<< "\n The contents of the queue are: \n";  
    display(q);  
    std::cout<< " ===== \n";  
  
    //empty  
    std::cout<< "\n is the queue empty? " << (q.empty() ? "Yes" :  
"No")<< "\n";  
    std::cout<< " ===== \n";  
  
    //size  
    std::cout<< "\n The size of the queue is " << (q.size())<< "\n";
```

```

std::cout<< " ===== \n";

//front
std::cout << "\n Front element: " << q.front() << "\n";
std::cout<< " ===== \n";

//back
std::cout << "\n Back element: " << q.back() << "\n";
std::cout<< " ===== \n";

//pop
q.pop();
std::cout << "\n After one pop:";
display(q);
std::cout<< " ===== \n";

//push
q.push('8');
std::cout << "\n After pushing '8': ";
display(q);
std::cout<< " ===== \n";

//display
std::cout << "\n Queue check: ";
display(q);
std::cout<< " ===== \n";

//deletion?
while (!q.empty()) {
    display(q);
    q.pop();
}
std::cout << "\n Final queue check: ";
display(q);
std::cout<< " ===== \n";
}

//function definitions
void display(std::queue<char> r) {
    // create a copy of the queue
    std::queue<char> c = r;

    while (!c.empty()) {
        std::cout << " " << c.front();
        c.pop();
    }
    std::cout << "\n";
}

```

OUTPUT:

```
"C:\Users\royga\CLionProjects\untitled1\H0A 5p1\ILOA.exe"
```

```
The contents of the queue are:
```

```
c p e 0 1 0
```

```
=====
```

```
is the queue empty? No
```

```
=====
```

```
The size of the queue is 6
```

```
=====
```

```
Front element: c
```

```
=====
```

```
Back element: 0
```

```
=====
```

```
After one pop: p e 0 1 0
```

```
=====
```

```
After pushing '8': p e 0 1 0 8
```

```
=====
```

```
Queue check: p e 0 1 0 8
```

```
=====
```

```
p e 0 1 0 8
```

```
e 0 1 0 8
```

```
0 1 0 8
```

```
1 0 8
```

```
0 8
```

```
8
```

```
Final queue check:
```

```
=====
```

```
Process finished with exit code 0
```

Code Analysis:

ILO A is basically part where we still use the stl of the queue, in this part we use the standard **push()**, **pop()**, **empty()**, **size()**, **front()**, **back()** and the self made **display()**. This part basically shows the major **queue** operations. Overall, this is like the stack activity but with some modifications that fits the first in first out idea.

ILO B:

CODE:

RUNNER:

```
//  
// Created by royga on 8/4/2026.  
//  
#include <iostream>  
#include <queue>  
#include "queue11.h"  
template <typename T>  
void display(qNode<T>* frontPtr);  
  
int main() {  
    qNode<char>* front = nullptr;  
    qNode<char>* back = nullptr;  
  
    enqueue(&front, &back, 'R');  
    std::cout << "Front: " << front->data << " Back: " << back->data <<  
    "\n";  
    std::cout<< " ===== \n";  
    enqueue(&front, &back, 'O');  
    std::cout << "Front: " << front->data << " Back: " << back->data <<  
    "\n";  
    std::cout<< " ===== \n";  
    enqueue(&front, &back, 'Y');  
    std::cout << "Front: " << front->data << " Back: " << back->data <<  
    "\n";  
    std::cout<< " ===== \n";  
    dequeue(&front, &back);  
    std::cout << "After dequeue: ";  
    display(front);  
    std::cout<< " ===== \n";  
    std::cout << "Front element: " << top1st(front) << "\n";  
    dequeue(&front, &back);
```

```

std::cout<< " ===== \n";
std::cout << "The new front: " << toplst(front) << "\n";
std::cout<< " ===== \n";

return 0;
}

```

HEADER:

```

//
// Created by royga on 8/4/2026.
//

#ifndef UNTITLED1_QUEUELL_H
#define UNTITLED1_QUEUELL_H

template<typename T>
class qNode {
public:
    T data;
    qNode<T> *next;
};

//creating a new node:
template <typename T>
qNode<T>* new_Node(T newData) {
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

//pushing from the back
template <typename T>
void enqueue(qNode<T>** frontPtr, qNode<T>** backPtr, T newData) {
    //create a new node;
    qNode<T>* newNode = new_Node(newData);

    //inserting to an empty queue
    if ((*frontPtr) == nullptr && (*backPtr) == nullptr ) {
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
    }

    //inserting an item into a non empty queue
    //point the backPtr next to the newNode
    (*backPtr)->next = newNode;
    (*backPtr) = newNode;
}

template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr) {
    //create a temporary node to store the node to be deleted

```

```

qNode<T>* deleteNode = nullptr;
deleteNode = (*frontPtr);

//check if the queue is only 1 node
if((*frontPtr)->next==nullptr && (*backPtr)->next==nullptr) {
    (*frontPtr)= nullptr;
    (*backPtr) = nullptr;
    delete deleteNode;
    return;
}
//deleting of the node
(*frontPtr)= deleteNode->next;
delete deleteNode;
}

//display all elements in the list
template <typename T>
void display(qNode<T>* frontPtr) {
    qNode<T>* current = frontPtr;
    while (current != nullptr) {
        std::cout << current->data << " ";
        current = current->next;
    }
    std::cout << "\n";
}

//return the front element
template <typename T>
T toplst(qNode<T>* frontPtr) {
    if (frontPtr == nullptr) {
        throw std::runtime_error("Queue is empty");
    }
    return frontPtr->data;
}

#endif //UNTITLED1_QUEUELL_H

```

OUTPUT:

```

"C:\Users\royga\CLionProjects\untitled1\H0A 5p1\ILOB.exe"
Front: R Back: R
=====
Front: R Back: 0
=====
Front: R Back: Y
=====
After dequeue: 0 Y
=====
Front element: 0
=====
After dequeue, the new front: Y
=====

Process finished with exit code 0

```

CODE ANALYSIS:

In this part of the discussion or HOA, we are using or creating our own **header or template** instead of the standard template **std::queue** already built, to use a **Linked list implementation** of the **queue**.

In the header file, we are still using the standard method of our professor to make the creation of the nodes, we then proceed to the **pushing / queuing** where we have to input the new pushed element to the back of the list.

Next is, we did the **dequeue** where we have to remove the first element by first putting it into a temporary node to store, then we have to delete it along with the node.

For output, the **display function** traverses the linked list starting from the front pointer and prints each element until the end of the list is reached.

Additionally, a **top1st function** is provided to return the front element without removing it, throwing an error if the queue is empty.

Altogether, this code or activity shows the complete linked-list queue implementation with **enqueue**, **dequeue**, **display**, and **peek** (in stack idea) operations, giving a clear view of how queues can be managed manually at the nodes.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: None

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: None

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: None

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.



I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.